PROGRAMAÇÃO MOBILE

Professor Peter Rizzon

Grupo Uniftec
centro universitário e faculdades

Introdução ao Android Studio

Grupo Uniftec
centro universitário e faculdades

ANDROID STUDIO

- Ambiente de Desenvolvimento Integrado (*IDE*) oficial para o desenvolvimento de apps para Android;
- É baseado no *IntelliJ IDEA*;
- Suporta desenvolvimento com Kotlin e Java;
- Integra editor, emulador, depuração e build;

IntelliJ IDEA permite escrita de código sem complicações.

Possui uma abordagem não-intrusiva intuitiva para ajudá-lo a escrever, depurar, refatorar, testar e aprender o seu código.

https://www.jetbrains.com/idea/

Grupo Uniftec
centro universitário e faculdades

ANDROID STUDIO

- Editor inteligente de código
- Emulador Android
- Gerenciador de dispositivos
- Ferramentas de build e execução
- Debug
- Integração com testes
- Layout tools

ANDROID STUDIO

- Ferramentas de *lint* para detectar problemas de desempenho, usabilidade, compatibilidade com versões, entre outros;

- Compatibilidade com C++ e NDK;

- Compatibilidade integrada com o Google Cloud Platform, facilitando a integração do Google Cloud Messaging e do App Engine.;

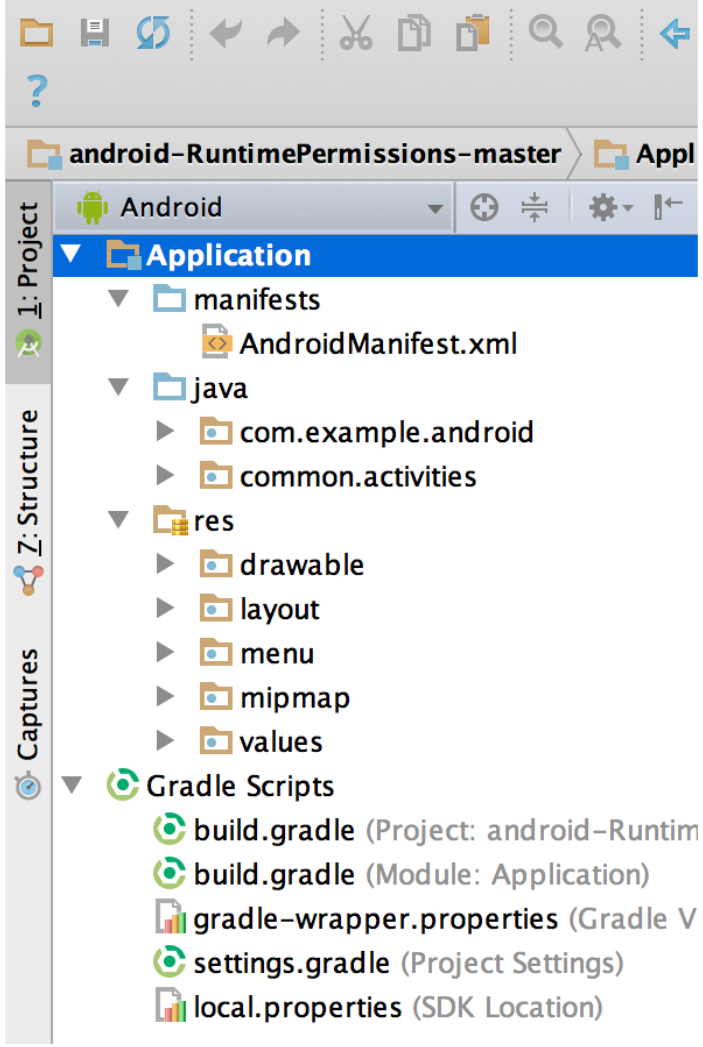
A ferramenta ***lint*** ajuda a encontrar códigos com estrutura ineficiente que podem afetar a confiabilidade e eficiência dos apps Android e dificultar a manutenção do código.

O Kit de Desenvolvimento Nativo (**NDK**) é um conjunto de ferramentas que permite usar códigos C e C++ no Android, além de fornecer bibliotecas de plataforma que podem ser usadas para gerenciar atividades nativas e acessar componentes físicos do dispositivo, como sensores e entrada de toque.

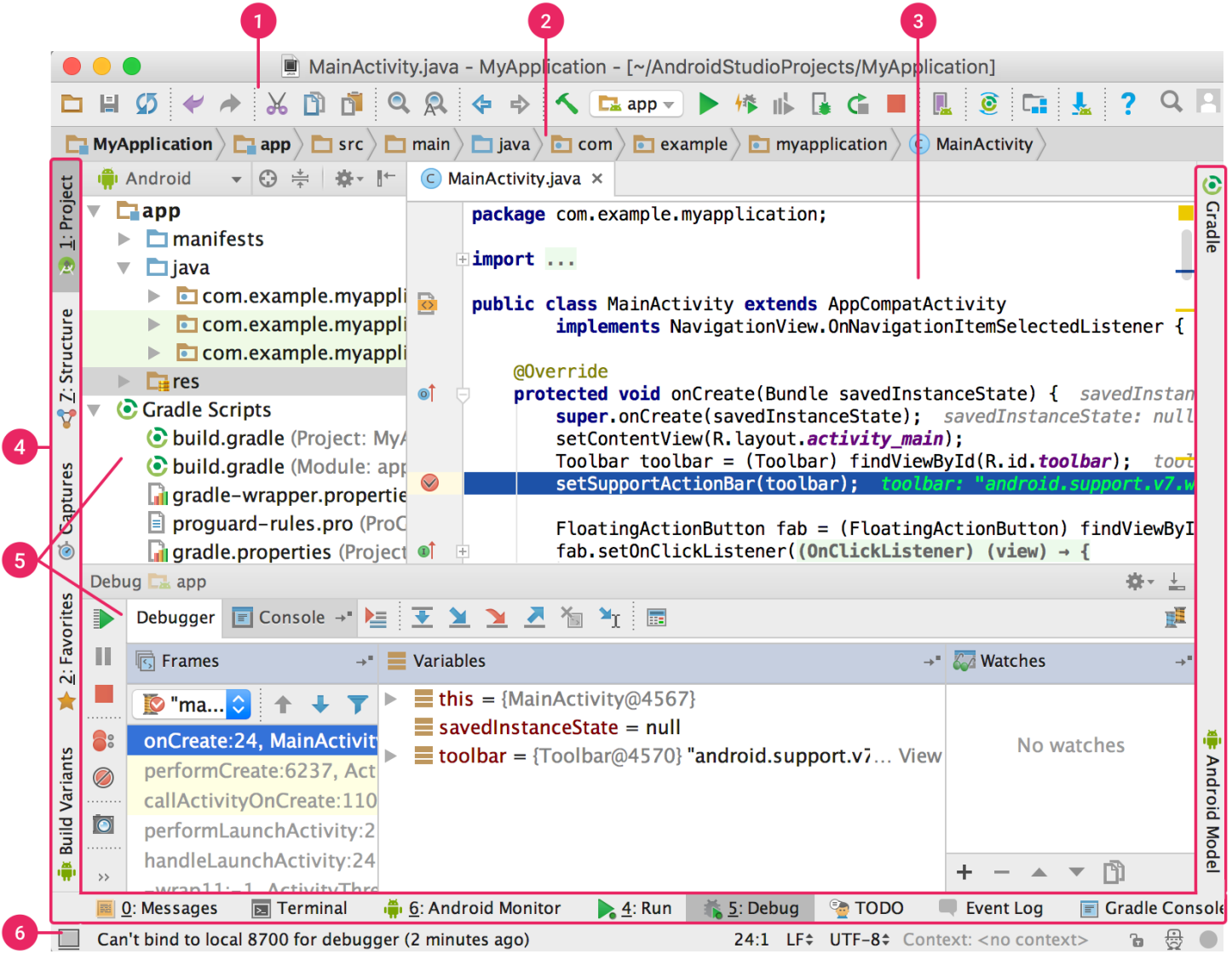
ANDROID STUDIO – ESTRUTURA DE UM PROJETO

- Cada projeto no Android Studio contém um ou mais módulos com arquivos de código-fonte e de recursos.

- Os tipos de módulos incluem:
  - Módulos de apps Android;
  - Módulos de biblioteca;
  - Módulos do Google App Engine;



ANDROID STUDIO – INTERFACE DE USUÁRIO

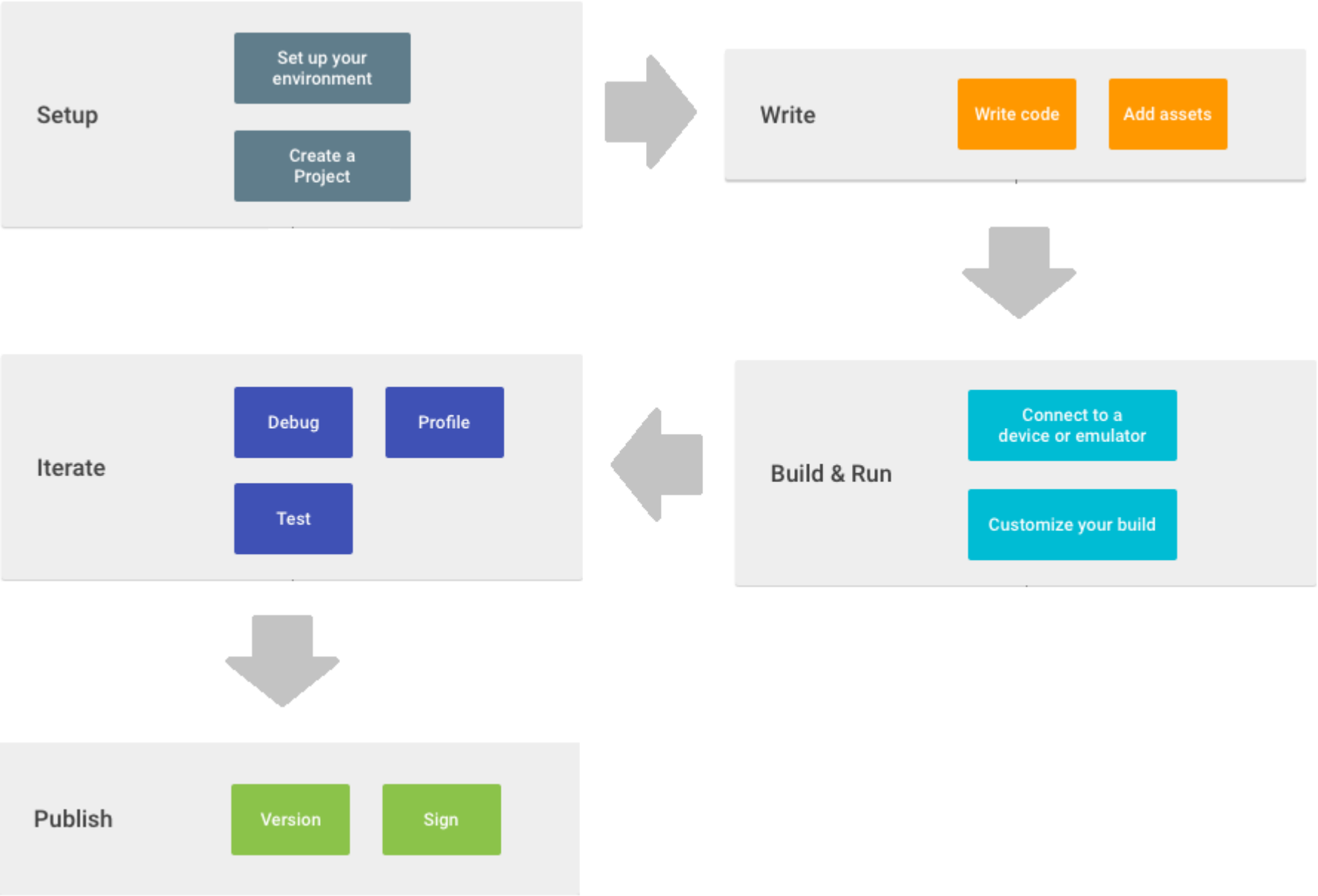


1. A **barra de ferramentas** permite realizar diversas ações, inclusive executar apps e inicializar ferramentas do Android.
2. A **barra de navegação** ajuda a navegar pelo projeto e a abrir arquivos para edição.
3. A **janela do editor** é onde você cria e modifica o código.
4. A **barra de janela de ferramentas** contém os botões que permitem expandir ou recolher as janelas de cada ferramenta.
5. As **janelas de ferramentas** permitem acessar tarefas específicas, como gerenciamento de projetos, pesquisa e controle de versões, entre outras.
6. A **barra de status** exibe o status do projeto e do próprio ambiente de desenvolvimento integrado, bem como todos os avisos ou mensagens.

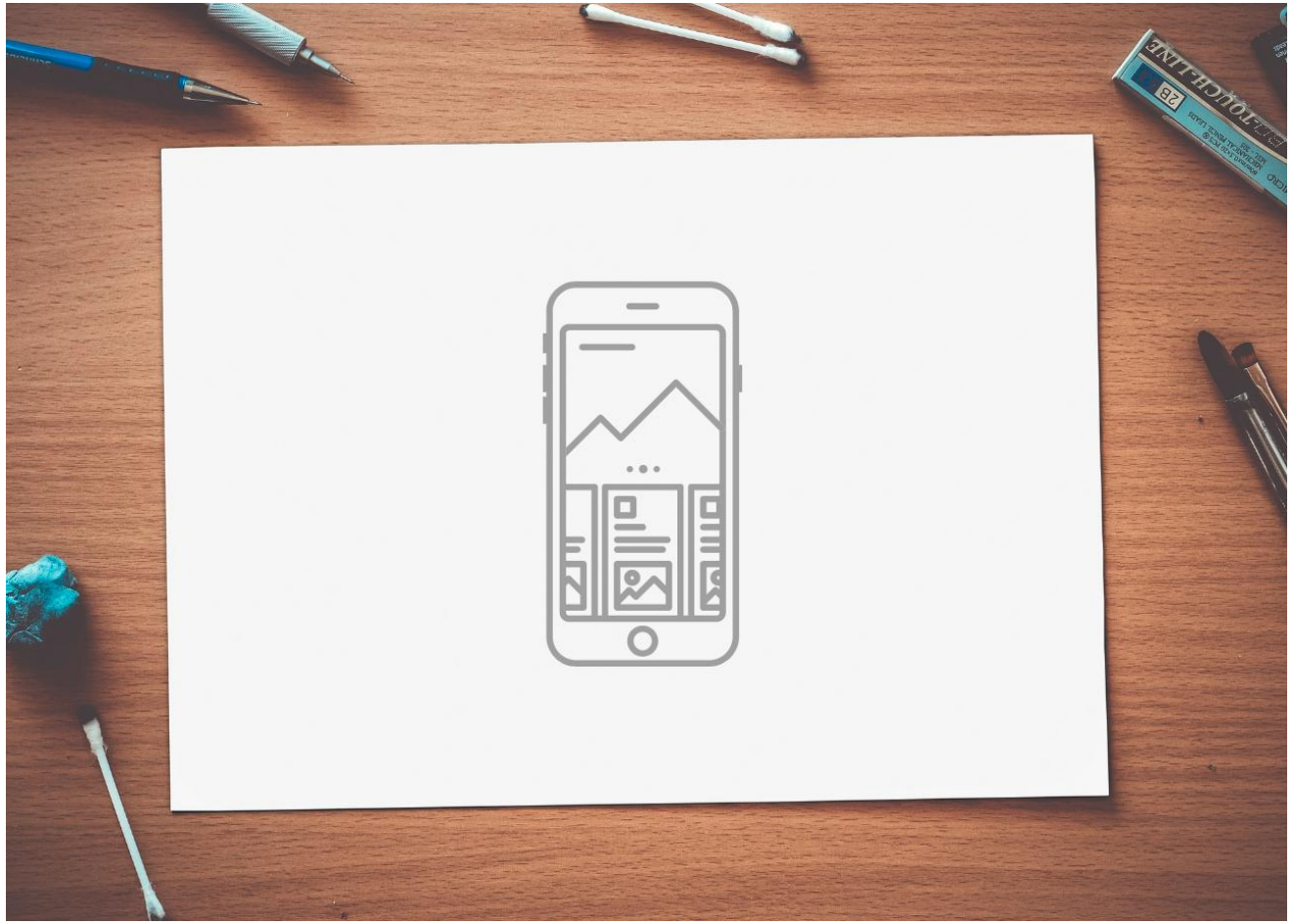
ANDROID STUDIO – PREENCHIMENTO AUTOMÁTICO DE CÓDIGO

Tipo	Descrição	Windows e Linux	Mac
Preenchimento básico	Exibe sugestões básicas para variáveis, tipos, métodos, expressões, entre outros. Se você chamar o preenchimento básico duas vezes seguidas, verá mais resultados, incluindo membros privados e membros estáticos não importados.	Control + Espaço	Control + Espaço
Preenchimento inteligente	Exibe opções relevantes de acordo com o contexto. O preenchimento inteligente detecta os tipos e os fluxos de dados esperados. Se você chamar o preenchimento inteligente duas vezes seguidas, verá mais resultados, incluindo cadeias.	Control + Shift + Espaço	Control + Shift + Espaço
Preenchimento de declaração	Preenche a declaração atual, adicionando parênteses, colchetes e chaves ausentes, formatação etc.	Control + Shift + Enter	Shift + Command + Enter

ANDROID STUDIO – FLUXO DE TRABALHO



Layouts no Android



Grupo Uniftec
centro universitário e faculdades

LAYOUTS NO ANDROID

- O layout define a estrutura de uma interface do usuário no aplicativo;
- A criação de interfaces no Android é realizada por arquivos .xml.

Grupo Uniftec
centro universitário e faculdades

LAYOUTS NO ANDROID

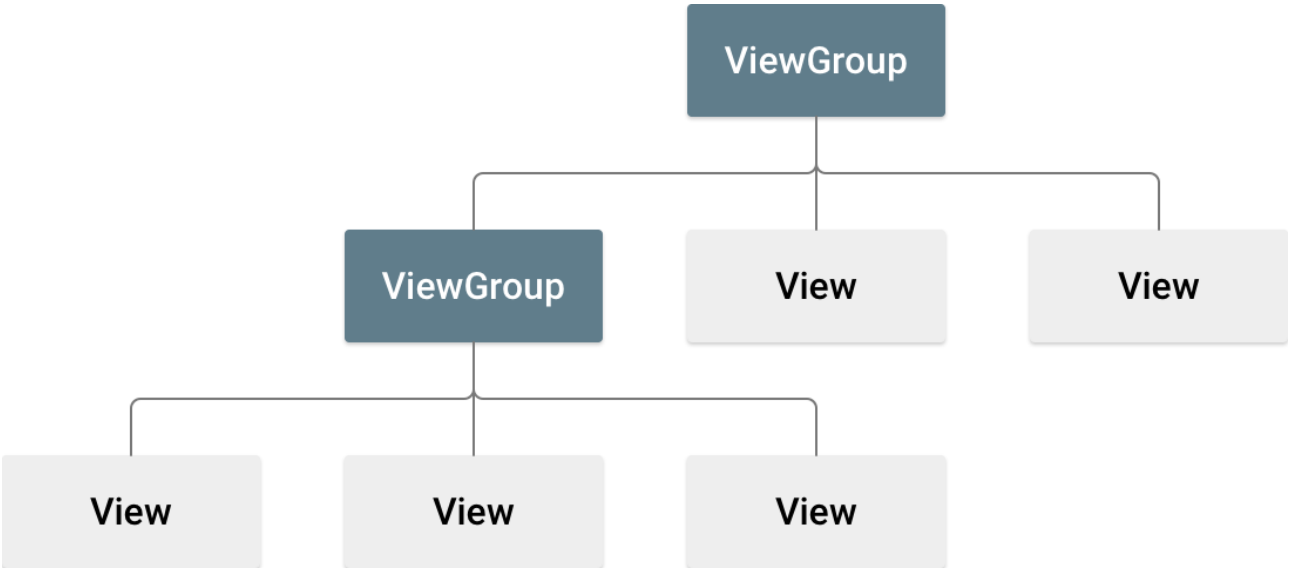
- Com o intuito de padronizar a construção de interfaces gráficas utilizadas para interagir com o usuário, o Android propõe uma terminologia que permite identificar categorias de componentes gráficos e as suas funções.

- Essa terminologia estabelece dois grupos de componentes gráficos fundamentados em classes específicas da API Android, sendo: ***Views*** e ***ViewGroups***.

LAYOUTS NO ANDROID

- A ***View*** geralmente desenha algo que o usuário pode ver e com que pode interagir. (*widgets*)

- Um ***ViewGroup*** é um container invisível que define a estrutura do layout para ***View*** e outros objetos ***ViewGroup.*** *(layouts)*



Grupo Uniftec
centro universitário e faculdades

LAYOUTS NO ANDROID - WIDGETS

- São subclasses diretas da classe View (android.view.View);
- Representam os componentes gráficos visuais dos aplicativos;
- Utilizados para a construção da interface gráfica das aplicações Android;
- Ex.: Botões (ImageButton, ToggleButton, RadioButton), selecionador de opções (CheckBoxes), caixas de texto, menus de itens, imagens, dentre outros.

Grupo Uniftec
centro universitário e faculdades

LAYOUTS NO ANDROID - LAYOUTS

- Os gerenciadores de layout são subclasses da classe ViewGroup (android.view.ViewGroup), que, por sua vez, é subclasse da classe View;
- ViewGroup é a base de todos os gerenciadores de layout presentes na plataforma Android.
- Representar os elementos gráficos não visuais, isto é, que são invisíveis para o usuário;
- Responsáveis por agrupar, organizar e controlar os componentes gráficos visuais, os widgets, da interface gráfica da aplicação;
- Ex.: LinearLayout e RelativeLayout.

Grupo Uniftec
centro universitário e faculdades

LAYOUTS NO ANDROID

Um layout pode ser declarado de duas maneiras:

- Declarar elementos da IU em XML. O Android fornece um vocabulário XML direto que corresponde às classes e subclasses de visualização, como as de widgets e layouts.

- Instanciar elementos do layout em ambiente de execução. O aplicativo pode criar objetos *View* e *ViewGroup* (e processar suas propriedades) programaticamente.

LAYOUTS NO ANDROID

```
<?xml version="1.0" encoding="utf-8"?>
<ViewGroup
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@[+][package:]id/resource_name"
    android:layout_height=["dimension" | "match_parent" | "wrap_content"]
    android:layout_width=["dimension" | "match_parent" | "wrap_content"]
    [ViewGroup-specific attributes] >
    <View
        android:id="@[+][package:]id/resource_name"
        android:layout_height=["dimension" | "match_parent" | "wrap_content"]
        android:layout_width=["dimension" | "match_parent" | "wrap_content"]
        [View-specific attributes] >
        <requestFocus/>
    </View>
    <ViewGroup >
        <View />
    </ViewGroup>
    <include layout="@layout/layout_resource"/>
</ViewGroup>
```

LAYOUTS NO ANDROID

```xml
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
              android:layout_width="match_parent"
              android:layout_height="match_parent"
              android:orientation="vertical" >
    <TextView android:id="@+id/text"
              android:layout_width="wrap_content"
              android:layout_height="wrap_content"
              android:text="Hello, I am a TextView" />
    <Button android:id="@+id/button"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Hello, I am a Button" />
</LinearLayout>
```

LAYOUTS NO ANDROID

Carregamento do recurso XML

- Carregar o recurso de layout na implementação de *callback Activity.onCreate().*
- Chamar *setContentView()*.
- Passar a referência para o recurso de layout na forma: *R.layout.layout_file_name*.

```
public void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.main_layout);
}
```

Grupo Uniftec
centro universitário e faculdades

LAYOUTS NO ANDROID

Parâmetros do Layout

- Os atributos do layout XML chamados *layout_something* definem parâmetros para a View apropriados para o ViewGroup em que reside.
- Todos os grupos de visualizações contêm largura e altura (*layout_width* e *layout_height*):
  - ***wrap_content:*** instrui a visualização a se redimensionar de acordo com as medidas exigidas pelo conteúdo.
  - ***match_parent:*** instrui a visualização a assumir o maior tamanho permitido pelo grupo de visualizações pais.
  - **medidas exatas**: Não recomendado**.**

LAYOUTS NO ANDROID – LINEAR LAYOUT

- Pode posicionar as Views em uma única coluna vertical ou horizontal.

LAYOUTS NO ANDROID – LINEAR LAYOUT

```xml
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
              android:layout_width="match_parent"
              android:layout_height="match_parent"
              android:orientation="vertical" >
    <TextView android:id="@+id/text"
              android:layout_width="wrap_content"
              android:layout_height="wrap_content"
              android:text="Hello, I am a TextView" />
    <Button android:id="@+id/button"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="Hello, I am a Button" />
</LinearLayout>
```

Grupo Uniftec
centro universitário e faculdades

LAYOUTS NO ANDROID – LINEAR LAYOUT

Ponderação do layout

- android:layout_weight.
- Confere um valor de “importância” a uma visualização em relação à quantidade de espaço que ela deve ocupar na tela.
- Um valor de ponderação maior permite que ela se expanda para preencher qualquer espaço restante na visualização pai.

Grupo Uniftec
centro universitário e faculdades

LAYOUTS NO ANDROID – LINEAR LAYOUT

Posição no layout

- android:layout_gravity.
- Controla a posição e o comportamento da View dentro de uma célula.

LAYOUTS NO ANDROID – LINEAR LAYOUT

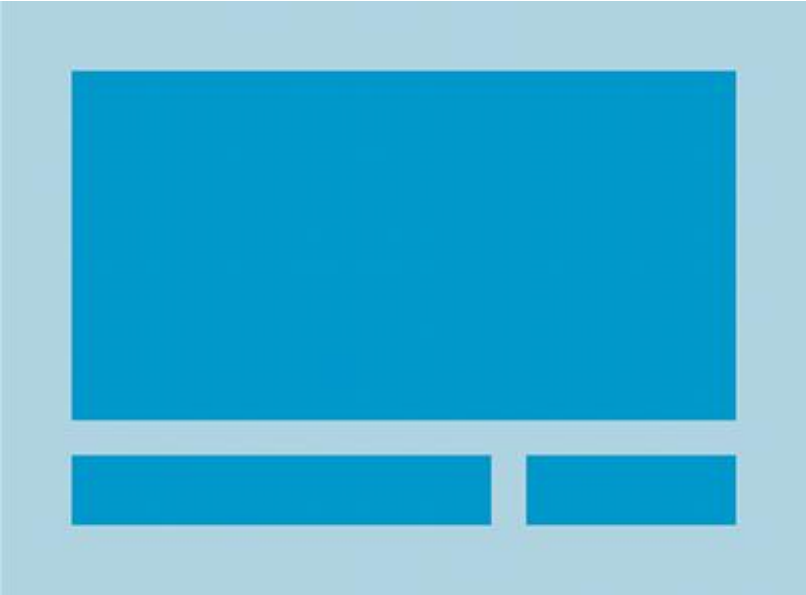
```xml
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:paddingLeft="16dp"
    android:paddingRight="16dp"
    android:orientation="vertical" >
    <EditText
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="@string/to" />
    <EditText
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="@string/subject" />
    <EditText
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1"
        android:gravity="top"
        android:hint="@string/message" />
    <Button
        android:layout_width="100dp"
        android:layout_height="wrap_content"
        android:layout_gravity="right"
        android:text="@string/send" />
</LinearLayout>
```

LAYOUTS NO ANDROID – RELATIVE LAYOUT

- O RelativeLayout (android.widget.RelativeLayout) permite posicionar componentes gráficos ao lado, abaixo ou acima de outro componente gráfico previamente adicionado ao layout.

- é um ViewGoup que exibe Views filhas em posições relativas.

- permite que as Views filhas especifiquem a posição delas em relação à Views pai ou entre si.



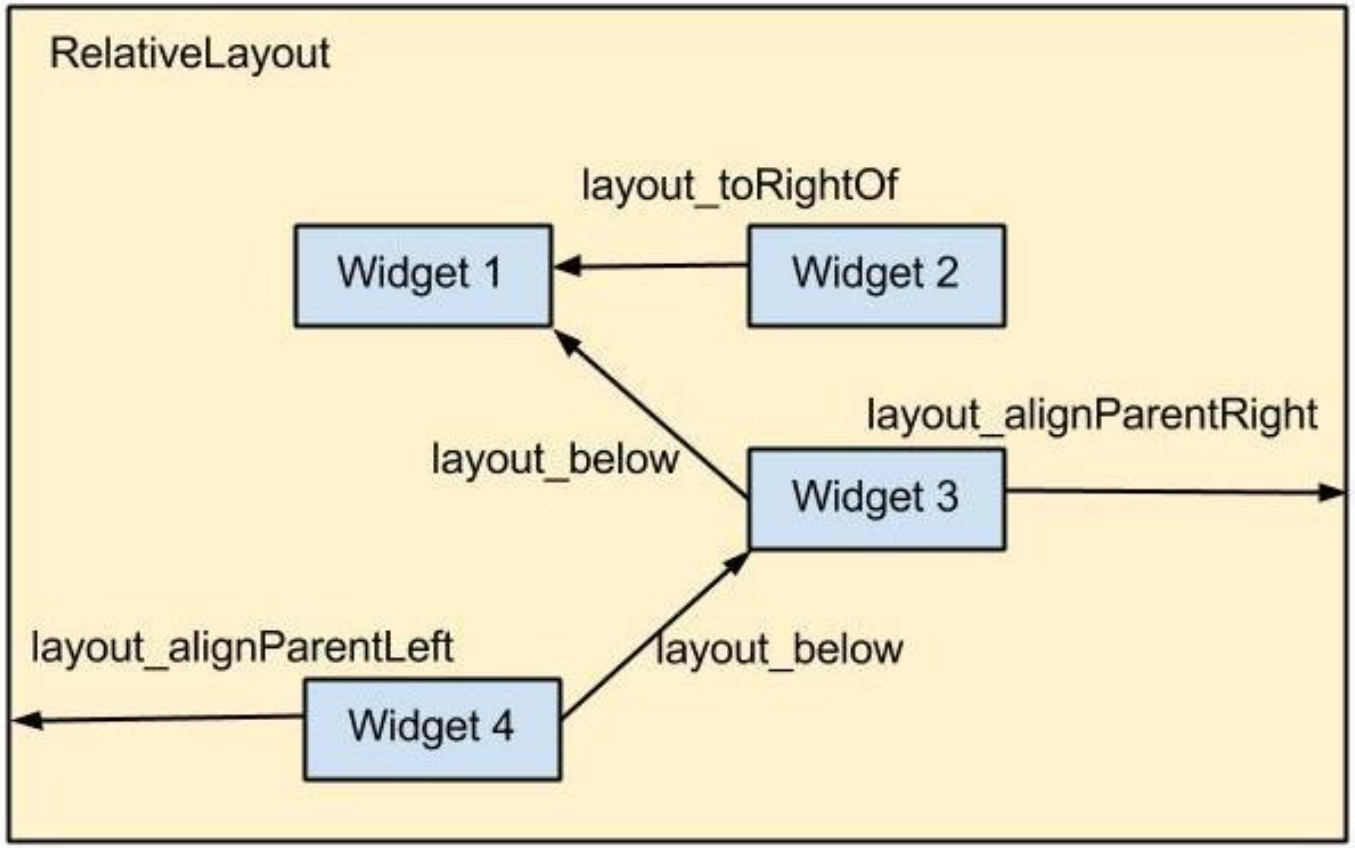
LAYOUTS NO ANDROID – RELATIVE LAYOUT

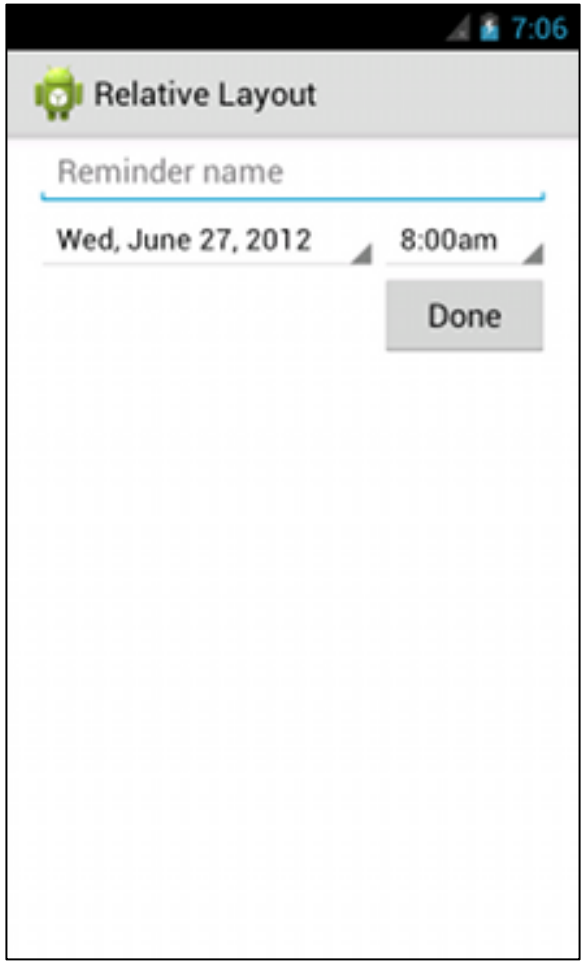
- Para que o posicionamento relativo seja feito:
  - é preciso que cada componente gráfico possua um identificador;
  - o componente gráfico utilizado como referência para os outros apareça antes na descrição do layout dentro do arquivo XML.

LAYOUTS NO ANDROID – RELATIVE LAYOUT

- Atributos em relação à View Pai:
  - android:layout_alignParentTop=“true/false”
  - android:layout_alignParentBottom=“true/false”
  - android:layout_alignParentLeft=“true/false”
  - android:layout_alignParentRight=“true/false”
  - android:layout_centerHorizontal=“true/false”
  - android:layout_centerVertical=“true/false”
- Atributos em relações entre views:
  - android:layout_toLeftOf=“@id/id_da_view”
  - android:layout_toRightOf=“@id/id_da_view”
  - android:layout_above=“@id/id_da_view”
  - android:layout_below=“@id/id_da_view”

LAYOUTS NO ANDROID – RELATIVE LAYOUT



LAYOUTS NO ANDROID – RELATIVE LAYOUT

```xml
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:paddingLeft="16dp"
    android:paddingRight="16dp" >
    <EditText
        android:id="@+id/name"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="@string/reminder" />
    <Spinner
        android:id="@+id/dates"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_below="@id/name"
        android:layout_alignParentLeft="true"
        android:layout_toLeftOf="@+id/times" />
    <Spinner
        android:id="@id/times"
        android:layout_width="96dp"
        android:layout_height="wrap_content"
        android:layout_below="@id/name"
        android:layout_alignParentRight="true" />
    <Button
        android:layout_width="96dp"
        android:layout_height="wrap_content"
        android:layout_below="@id/times"
        android:layout_alignParentRight="true"
        android:text="@string/done" />
</RelativeLayout>
```